

Inter-Process Communication (IPC)

1. Introduction :

What is IPC?

IPC (Inter-Process Communication) is a mechanism that allows two or more processes to **exchange data and synchronize** their activities.

Normally, each process has its own separate memory space. One process cannot directly access another process's memory. IPC provides a safe way for processes to communicate with each other.

Why Do We Need IPC?

Consider the following examples:

- A browser process communicates with a rendering process.
- A media player communicates with an audio service.
- An Android application communicates with system services.
- A parent process sends data to a child process.

Without IPC, processes would work independently and could not share information.

2. Need for IPC

IPC is used for:

- Data Sharing.
- One process sends data to another process.

Example:

- Sensor process collects temperature.
- Display process shows temperature.

Resource Sharing

Multiple processes access common resources.

Example:

- Multiple applications use the same printer.
-

Types of IPC

Linux provides several IPC mechanisms.

1. Pipe

Definition

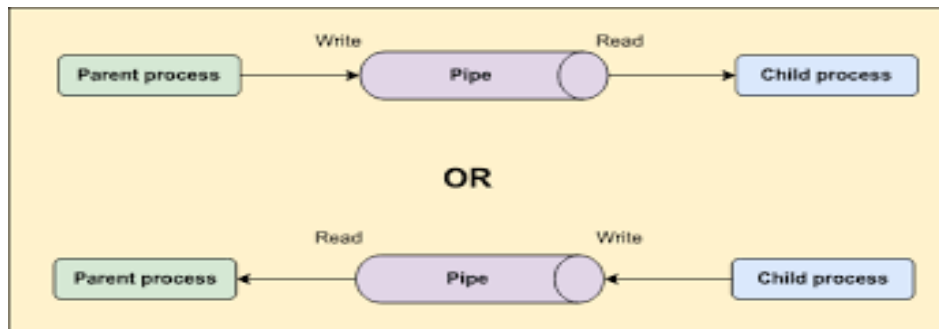
A pipe is the simplest IPC mechanism.

It creates a communication channel between related processes.

Characteristics

- Half-duplex communication
- Usually between parent and child processes
- Data flows in one direction

Diagram



System Call

`pipe(fd);`

Advantages

- Simple
- Fast

Limitations

- Only for related processes
- One-way communication

2. Named Pipe (FIFO)

Definition

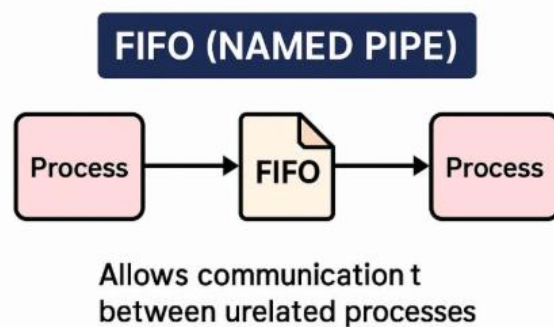
A FIFO is a special file used for communication.

Unlike ordinary pipes, unrelated processes can communicate.

Creation

`mkfifo("myfifo", 0666);`

Diagram



Advantages

- Works between unrelated processes

Limitations

- Slower than shared memory
-

3. Message Queue

Definition

Processes exchange messages through a queue maintained by the kernel.

Diagram



System Calls

msgget()
msgsnd()
msgrcv()
msgctl()

Advantages

- Messages have priorities
- No need for synchronization of data structure

Limitations

- Kernel copying overhead

Example

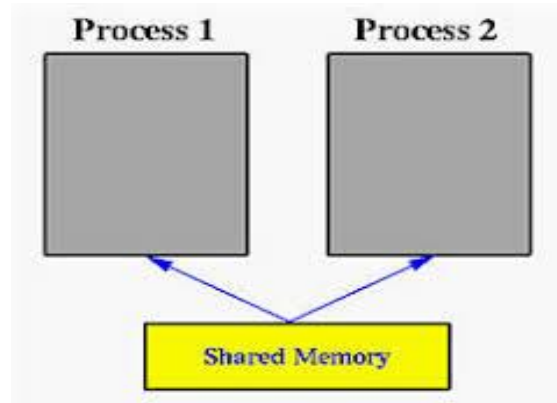
Chat application
Task scheduling system

4. Shared Memory

Definition

Shared memory allows multiple processes to access the same memory region.

Diagram



System Calls

shmget()

shmat()

shmdt()

shmctl()

Advantages

- Fastest IPC mechanism
- No repeated kernel copying

Limitations

- Requires synchronization
- Risk of data corruption

Example

Video processing

Database applications

High-speed data transfer

5. Semaphore

Definition

A semaphore is used for synchronization between processes.

It controls access to shared resources.

Types

Binary Semaphore

Values:

0 or 1

Works like a lock.

Counting Semaphore

Values:

0,1,2,3,...

Controls multiple resources.

Operations

wait() -> P operation

signal() -> V operation

Example

Printer access control

Producer-Consumer problem

Advantages

- Prevents race conditions

Limitations

- Incorrect usage may cause deadlock